

Section A: Scenario-Based Questions

For each scenario, decide Relational or NoSQL (and which NoSQL type, if applicable), then justify your choice in 2- 3 sentences using the concepts from the session (ACID/BASE, schema flexibility, query pattern).

A1. Payment Processing System

A fintech app needs to debit one account and credit another atomically, with strict rollback if either step fails.

Relational Database: It has a fixed schema that ensures the structure if rollbacks are necessary. It also has ACID principles guarantees the transactions

A2. E-commerce Product Catalog

Products range from books to electronics to furniture — each category has wildly different attributes (e.g., "page count" vs "battery life" vs "dimensions").

NoSQL, document type. Each product range could be a different document with their own structure.

A3. Social Media Friend Recommendations

The app needs to suggest "people you may know" by finding friends-of-friends up to 3 levels deep.

NoSQL, Graph. It connects data and has relationships.

A4. IoT Sensor Network

Thousands of sensors report temperature readings every second; the system must handle very high write throughput, and reads are mostly "give me the last 24 hours for sensor X."

NoSQL, Column Family. It is excellent for high-scale writes and time-series read patterns.

A5. Session/Cart Management

A retail site needs to store each logged-in user's shopping cart for fast retrieval by session ID, with no complex querying required.

NoSQL, Key-Value. Each session is the key and their information is in the value. It is extremely fast looking up by key

A6. Financial Reporting Dashboard

A finance team needs to join customer, order, and payment tables to produce monthly revenue reports with strict accuracy.

Relational database. For joining tables it is necessary that the tables have primary and foreign key.

Section B: Multiple Choice Questions

B1. What does a foreign key primarily enforce?

- a) Query performance
- b) Referential integrity between tables

- c) Schema flexibility
- d) Data compression

B2. Which statement about NoSQL is most accurate?

- a) NoSQL means no schema at all
- b) NoSQL databases can never scale
- c) NoSQL favors flexible schema over strict enforcement
- d) NoSQL databases don't support any data validation

B3. In the ACID model, "Isolation" refers to:

- a) Data being stored on isolated servers
- b) Concurrent transactions not interfering with each other
- c) Backups being isolated from production
- d) Tables being isolated from each other

B4. Which NoSQL type is best suited for "friends of friends" traversal queries?

- a) Key-value
- b) Document
- c) Column-family
- d) Graph

B5. BASE stands for:

- a) Basic Availability, Stateless Execution
- b) Basically Available, Soft state, Eventual consistency
- c) Balanced Access, Secure Encryption
- d) Batch Aggregation, Synchronous Execution

B6. A primary key automatically implies:

- a) The column is encrypted
- b) An index is often created for uniqueness enforcement
- c) The table has no foreign keys
- d) The column must be a string type

B7. Which use case is the best fit for a column-family database?

- a) Bank account transfers
- b) Per-user daily time-series metrics with many optional fields
- c) Storing a single user's session token
- d) Modeling friendships between users

B8. What is a key trade-off when choosing BASE over ACID?

- a) You gain strong consistency but lose availability
- b) You gain availability/partition tolerance but accept eventual consistency
- c) You gain faster joins but lose indexing
- d) You gain schema enforcement but lose flexibility

B9. Embedding line items inside an order document (vs. referencing them) is a pattern typical of:

- a) Column-family databases
- b) Graph databases
- c) Document databases
- d) Key-value databases

B10. Which scenario is the weakest fit for a graph database?

- a) Fraud ring detection across connected accounts
- b) Mutual friend discovery
- c) Large-scale tabular financial reporting with millions of rows
- d) Recommendation engines based on relationships

Section C: Theory Questions

C1. Explain the difference between a primary key and a foreign key, and describe one real-world consequence of missing a foreign key constraint.

The primary key is the field of a table that for every register is unique, it works like an identifier. The foreign key is the field that referees to the primary key of another table, it keeps the relationship between tables.

If a foreign key is missing this table can't refer to others, only can be referred to by other tables that have as foreign key his primary key. For example: orders table has order_id, amount and date and clients table has customer_id, customer_name, order_id. The order table can't refer to other tables because it hasn't a foreign key. But can be referred to by the clients table, because it has as foreign key the orders primary key, order_id.

C2. Compare ACID and BASE across consistency, availability, and typical use cases. Give one example use case for each that was not used in the session notes.

ACID prevails the correctness and BASE the availability. For that reason the consistency in ACID is higher than in BASE, but BASE has more availability of the data than ACID during partitions.

ACID use case: birth registration

BASE use case: sensor records

C3. "NoSQL means no schema." Explain why this is a common misconception and describe what actually happens to schema in a NoSQL system.

NoSQL has a flexible schema. This is useful for heterogeneous data, it has a pre decfined structure, but if required it can be changed.

C4. Describe the four main NoSQL types (document, key-value, column-family, graph) in one sentence each, and give one data engineering example per type.

Document: a collection of documents, where each document is like a JSON file. Collection is equivalent to table in relational database and document to row. For example: product collection, each document is a product with their specific attributes.

Key-value: every record has a unique key and the values can be different for each record. For example: Web or app session recording.

Column-family: rows are primary keys grouped by columns. Used for vast amount of data and low latency. For example: ecommerce events, each event_id is a primary key with group of columns such as event_type, page.

Graph: group of nodes and edges that have connections. For example: social media users and their connection with other users.

C5.A team wants to migrate a fixed-schema product table to a flexible-schema document model because product attributes vary heavily by category. What is gained, and what risk should the team watch for after migration?

They gain the flexibility to adapt to new attributes and enrich the database, but they also have the risk of losing data integrity.

Section D: Practical Questions (Try it yourself)

D1. Relational -Build and Query

Using sqlite3, create two tables: employees(employee_id PK, name, dept_id FK) and departments(dept_id PK, dept_name). Insert at least 2 departments and 3 employees. Write a query that joins both tables to list each employee's name alongside their department name.

```
SELECT
    e.name,
    d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id=d.dept_id;
```

D2. ACID - Transaction Rollback

Using sqlite3, create a wallet(user, balance) table. Simulate a transfer of 30 units from one user to another inside a transaction. Deliberately raise an exception before committing, then roll back and print the table to confirm balances are unchanged.

```
import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

# CREATE TABLE
cursor.execute("""
```

```
CREATE TABLE wallet (  
    user TEXT,  
    balance INTEGER  
)  
""")  
conn.commit()  
# INSERT DATA  
cursor.execute("""  
INSERT INTO wallet VALUES ('Vicky', 100),('Alex',200)  
""")  
conn.commit()  
# SHOW WHAT HAS THE TABLE  
cursor.execute("SELECT * FROM wallet")  
print(cursor.fetchall())  
# TRANSACTIONS  
try:  
    # UPDATE VICKY  
    cursor.execute("""  
UPDATE wallet SET 'Balance'=70 WHERE user='Vicky'  
""")  
  
    # UPDATE ALEX  
    cursor.execute("""  
UPDATE wallet SET 'Balance'=230 WHERE user='Alex'  
""")  
  
    # raise simulated error during the transaction  
    raise Exception("Transaction error")
```

```

conn.commit()

except Exception as e:
    print(e)
    conn.rollback()

# PROOF THE ROLLBACK WAS OK

cursor.execute("SELECT * FROM wallet")

print(cursor.fetchall())

```

D3. Document Model - Variable Attributes

Using Python dictionaries, create a small in-memory "collection" (list of dicts) of at least 3 products from different categories (e.g., book, laptop, chair), where each product has different fields beyond a shared sku and name. Write a function that filters and returns only products that have a battery_life field.

```

product_collection = [{"name": "book", "sku": "123"}, {"name": "laptop",
"sku": "234", "battery_life": 70}, {"name": "chair", "sku": "345"},
{"name": "smartphont", "sku": "222", "battery_life": 10}]

```

```

def battery_life_products(products):
    collection = []
    for product in products:
        if "battery_life" in product:
            collection.append(product)
        else:
            continue
    return collection

battery_life_products(product_collection)

```

D4. Key-Value Store - Simulated Cache

Using a Python dict as a key-value store, write code to store 3 user sessions keyed by "session:<id>", then write a function `get_session(session_id)` that returns the session or `None` if not found (simulating a cache miss).

```
sessions
={ "session_1": {"user": "111"}, "session_12": {"user": "222"}, "session_3": {"user": "333"} }

def get_session(session_id):
    session_id_name = "session_"+str(session_id)

    if session_id_name in sessions:
        return sessions[session_id_name]
    else:
        print("None")
```

D5. Eventual Consistency

Using two Python dicts (`replicaA`, `replicaB`) to simulate replicas, write a value to `replicaA` only, show that reading it from `replicaB` returns nothing (stale/missing), then simulate a sync and show `replicaB` now has the value. Add a short comment explaining which BASE property this demonstrates.

```
replicaA = {"id": 1}
replicaB = {}

# Read before synchronization
if "id" in replicaB:
    print(replicaB["id"])
else:
    print("replicaB does not have the value yet.")

# Simulate synchronization
replicaB["id"] = replicaA["id"]

# Read after synchronization
print(replicaB["id"])
```

```
# This demonstrates Eventual Consistency:
# replicaB does not immediately see the update,
# but after synchronization both replicas contain the same value.
```

D6. Graph - Friends of Friends

Using an adjacency list (dict of lists) representing a small social graph with at least 4 users, write code to compute the friends-of-friends for one user, excluding the user themselves and their direct friends.

```
user_relations_list =
{"user1":["user2","user3"],"user2":["user1","user10"],"user3":["user1","user33",""
user10"],"user10":["user2"], "user33":["user3"]}

def seach_friends_of_friends(user_relations):
    friends_of_friends = {}
    for user in user_relations:
        friends_of_friends[user] = []
        for friend in user_relations[user]:
            friends_of_friend = user_relations[friend]
            for friend_of_friend in user_relations[friend]:
                if friend_of_friend != user:
                    friends_of_friends[user].append(friend_of_friend)
        friends_of_friends[user]=set(friends_of_friends[user])
    return friends_of_friends

seach_friends_of_friends(user_relations_list)
```